# Dynamic Programming

Solving Complex Problems with Optimal Substructure

# What is Dynamic Programming?

- **Breaking Down:** Decompose into overlapping subproblems
- **Memoization:** Cache results to avoid recomputation
- **Bottom-Up/Top-Down:** Tabulation or recursion with caching
- **Optimal Substructure:** Solution combines optimal subproblem solutions

# Optimal Substructure

**Definition:** An optimal solution contains optimal solutions to subproblems.

- **Example:** Shortest path A→D = shortest path A→C + shortest path C→D
- If you have optimal solution to problem, subproblems within it must be optimal
- This property enables DP to work correctly

# Overlapping Subproblems

Same subproblems solved multiple times in naive recursion.

### Without DP (Naive)

fib(5) calls fib(3) twice

Exponential: O(2^n)

### With DP (Memoization)

Compute fib(3) once, reuse

Polynomial: O(n)

# DP Approaches: Top-Down vs Bottom-Up

## Top-Down

Recursion + Memoization

Solves as needed, intuitive

## Bottom-Up

Iteration + Tabulation

Solves all states, efficient

# Essential Graph Algorithms

Finding shortest paths between vertices with varying constraints:

- **Dijkstra:** Single-source, non-negative weights
- **Bellman-Ford:** Single-source, handles negative weights
- **Floyd-Warshall:** All-pairs shortest paths, negative weights allowed

# Bellman-Ford Algorithm

Relaxes all edges |V|-1 times, detects negative cycles.

- **Time Complexity:** O(VE)
- **Space:** O(V)
- **Advantage:** Works with negative weights, detects negative cycles
- **Use Case:** Currency arbitrage, network routing with losses

# Bellman-Ford: Distance Updates

A=0, B=4, C=2, D=7, E=5

# Floyd-Warshall Algorithm

DP approach for all-pairs shortest paths. Iteratively considers intermediate vertices.

- **Time Complexity:** $O(V^3)$
- **Space:** $O(V^2)$ for distance matrix
- **DP Recurrence:** dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])

# Floyd-Warshall: Distance Matrix Evolution

Initial (Direct Edges)

| | A | B | C |
|---|---|---|---|
| **A** | 0 | 4 | ∞ |
| **B** | ∞ | 0 | 2 |
| **C** | 1 | ∞ | 0 |

After k=2 (Final)

| | A | B | C |
|---|---|---|---|
| **A** | 0 | 3 | 5 |
| **B** | 3 | 0 | 2 |
| **C** | 1 | 5 | 0 |

# Bellman-Ford vs Floyd-Warshall

## Bellman-Ford

**Single-source** shortest paths

O(VE) | Detects cycles

## Floyd-Warshall

**All-pairs** shortest paths

$O(V^3)$ | Pure DP approach

# Why DP for Graph Algorithms?

- **Optimal Substructure:** Shortest path A→D through C is built from A→C and C→D
- **Overlapping Subproblems:** Multiple paths compute shortest A→C multiple times
- **Memoization Benefit:** Cache distances to avoid recomputation
- **Floyd-Warshall Example:** Considers all intermediate nodes k as potential shortcuts

# Shortest Path Algorithms Comparison

**Dijkstra**

$O((V+E)\log V)$ | Non-neg only

**Bellman-Ford**

$O(VE)$ | Neg weights OK

**Floyd-Warshall**

$O(V^3)$ | All pairs, DP

**BFS/Unweighted**

$O(V+E)$ | Single source

# Key DP Properties & When to Use

- **Optimal Substructure:** Solution from optimal sub-solutions (Bellman-Ford relaxation)
- **Overlapping Subproblems:** Recompute same distances (memoize with table)
- **Memoization vs Tabulation:** Choose based on state access pattern
- **Verify:** Not all greedy problems have optimal substructure!

# Real-World Applications

- **GPS Navigation:** Dijkstra or A* for fastest route
- **Currency Exchange:** Bellman-Ford detects arbitrage (negative cycles)
- **Network Routing:** Floyd-Warshall for all-node communication
- **Game AI:** DP for optimal move sequences with memoization
- **Sequence Alignment:** DP for DNA/protein matching (Bioinformatics)

# Thank You!